

02-select-fewshot-examples

September 29, 2025

0.0.1 Setup and Load Data

This cell sets up the environment and loads the `hard_examples.json` file generated by the analysis notebook.

```
[2]: # CELL 1: Setup and Load Data
import json
import pandas as pd
from pathlib import Path

# --- CONFIGURATION ---
# Automatically find the latest pipeline run via the pointer file
OUTPUT_ROOT = Path("./data/derived")
latest_run_ptr = OUTPUT_ROOT / "psycomark_latest.txt"

if not latest_run_ptr.exists():
    raise FileNotFoundError(f"Could not find pointer file at '{latest_run_ptr}'.
    ↳ Please run data_pipeline.py first.")

PIPELINE_OUTPUT_DIR = Path(latest_run_ptr.read_text().strip())
print(f"--- Loading data from latest pipeline run: {PIPELINE_OUTPUT_DIR.name}
    ↳ ---")

# --- Load the 'hard_examples.json' and the full dev set for context ---
hard_examples_path = PIPELINE_OUTPUT_DIR / "hard_examples.json"
if not hard_examples_path.exists():
    raise FileNotFoundError(f"File '{hard_examples_path}' not found. Please run
    ↳ the analysis notebook first.")

with open(hard_examples_path, 'r', encoding='utf-8') as f:
    hard_examples = json.load(f)

# Load the full dataset to fetch marker and label info
train_df = pd.read_json(PIPELINE_OUTPUT_DIR / "train.jsonl", lines=True)
dev_df = pd.read_json(PIPELINE_OUTPUT_DIR / "dev.jsonl", lines=True)
df_all = pd.concat([train_df, dev_df], ignore_index=True).set_index('doc_id')

# Convert to a DataFrame for easier processing
```

```
df_hard = pd.DataFrame(hard_examples).set_index('doc_id')
df_hard = df_hard.join(df_all[['markers', 'doc_label']])

print(f"Loaded {len(df_hard)} candidate 'hard' examples.")
df_hard.head()
```

```
--- Loading data from latest pipeline run:
psycomark_official_split_20250928_232947 ---
Loaded 1167 candidate 'hard' examples.
```

```
[2]:
```

	reasons \
doc_id	
t1_csthti6	[High Action/Effect IoU (1.00)]
t1_dp0t655	[High Action/Effect IoU (1.00)]
t1_dt9elm2	[High Action/Effect IoU (1.00), High Subreddit...
t1_dtqffjs	[High Action/Effect IoU (1.00), High Subreddit...
t1_dw374sc	[High Action/Effect IoU (1.00)]

	text \
doc_id	
t1_csthti6	Saw this in /r/SandersForPresident and they se...
t1_dp0t655	Since my last post sparked a heated discussi...
t1_dt9elm2	Wasserman-Shultz clearly stole the election in...
t1_dtqffjs	Starts around 11:45. Swalwell confirms they ma...
t1_dw374sc	Just as Net Neutrality was meant to keep any o...

	markers	doc_label
doc_id		
t1_csthti6	[{'label': 'Evidence', 'start': 12, 'end': 34, ...	non
t1_dp0t655	[{'label': 'Action', 'start': 31, 'end': 49, '...	non
t1_dt9elm2	[{'label': 'Actor', 'start': 0, 'end': 16, 'te...	conspiracy
t1_dtqffjs	[{'label': 'Actor', 'start': 39, 'end': 43, 't...	cant_tell
t1_dw374sc	[{'label': 'Actor', 'start': 41, 'end': 56, 't...	cant_tell

0.0.2 Define a Scoring Strategy to Rank Examples

Here, we define a function to score each example. The score will be based on how well an example fulfills our desired criteria: demonstrating ambiguity, covering different labels, and having a reasonable length.

```
[3]: # CELL 2: Define a Scoring Strategy to Rank Examples

def score_example(row):
    """
    Scores a candidate few-shot example based on a set of heuristics.
    Higher score is better.
    """
    score = 0
```

```

reasons = row['reasons']

# --- Criterion 1: Ambiguity and Complexity ---
# High score for examples that are hard for multiple reasons
score += len(reasons) * 10

# Specific bonus for high Action/Effect or Actor/Victim overlap
if any("High Action/Effect IoU" in r for r in reasons):
    score += 15
if any("High Actor/Victim IoU" in r for r in reasons): # We'll need to add_
↪this to the hard examples script later
    score += 15

# --- Criterion 2: Label Coverage ---
# Give points for having a clear conspiracy or non-conspiracy label
if row['doc_label'] == 'conspiracy':
    score += 10
elif row['doc_label'] == 'non':
    score += 10
# No points for 'cant_tell' as it's less instructive

# --- Criterion 3: Marker Density ---
# More markers are generally more instructive, up to a point
num_markers = len(row['markers']) if isinstance(row['markers'], list) else 0
score += min(num_markers, 5) * 2 # Add up to 10 points for marker density

# Bonus for having a diverse set of marker labels
if num_markers > 0:
    unique_labels = {m['label'] for m in row['markers']}
    score += len(unique_labels) * 3

# --- Criterion 4: Text Length ---
# Penalize very short or very long examples. Sweet spot is 200-600 chars.
text_len = len(row['text'])
if 200 < text_len < 600:
    score += 5
elif text_len < 160 or text_len > 800:
    score -= 10

return score

# Apply the scoring function
df_hard['score'] = df_hard.apply(score_example, axis=1)

# Display the top-scoring examples
df_hard_sorted = df_hard.sort_values('score', ascending=False)
print("Top 10 highest-scoring few-shot candidates:")

```

```
display(df_hard_sorted[['reasons', 'doc_label', 'score', 'text']].head(10))
```

Top 10 highest-scoring few-shot candidates:

doc_id	reasons	doc_label	\
t1_f7bzvwr	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_e1pv6xg	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_dt9elm2	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_eac1hkd	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_gb06koo	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_f76erur	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_erkidj3	[High Action/Effect IoU (0.78), High Subreddit...	conspiracy	
t1_elcq7w6	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_jcxpg2x	[High Action/Effect IoU (1.00), High Subreddit...	conspiracy	
t1_f3otibj	[High Action/Effect IoU (1.00), High Subreddit...	non	

doc_id	score	text
t1_f7bzvwr	75	World War One killed millions of people, destr...
t1_e1pv6xg	75	Amy Manson, actress in the upcoming Doom movie...
t1_dt9elm2	75	Wasserman-Shultz clearly stole the election in...
t1_eac1hkd	75	how many more Russians are going to get poison...
t1_gb06koo	75	Your tax exempt cult is a poison on these land...
t1_f76erur	75	Perhaps Shokin was not a great prosecutor gene...
t1_erkidj3	75	I think this ironic comment shows that the imm...
t1_elcq7w6	72	Israel's backers will shamelessly exploit any ...
t1_jcxpg2x	72	In 1987, an elderly couple reported human bloo...
t1_f3otibj	72	I try not clog up this forum with stuff off r/...

0.0.3 Select the Best N Examples and Export

This final cell selects the best examples based on our scoring. It ensures we have a balanced set (e.g., at least one ‘conspiracy’ and one ‘non’) and then exports them to a clean JSON file.

```
[4]: # CELL 3: Select the Best N Examples and Export

# --- Selection Strategy ---
N_EXAMPLES_TO_SELECT = 3

selected_examples = []
selected_doc_ids = set()
seen_labels = set()

df_selection_pool = df_hard_sorted.copy()

# --- Step 1: Ensure we have at least one 'conspiracy' and one 'non' example if possible ---
```

```

best_conspiracy = df_selection_pool[df_selection_pool['doc_label'] == 'conspiracy'].iloc[0:1]
best_non = df_selection_pool[df_selection_pool['doc_label'] == 'non'].iloc[0:1]

if not best_conspiracy.empty:
    selected_doc_ids.add(best_conspiracy.index[0])
    seen_labels.add('conspiracy')

if not best_non.empty:
    selected_doc_ids.add(best_non.index[0])
    seen_labels.add('non')

# --- Step 2: Fill the remaining slots from the top of the general pool ---
remaining_needed = N_EXAMPLES_TO_SELECT - len(selected_doc_ids)
if remaining_needed > 0:
    # Exclude already selected docs
    pool_without_selected = df_selection_pool.drop(index=list(selected_doc_ids))

    # Fill with the next best examples
    for doc_id, row in pool_without_selected.head(remaining_needed).iterrows():
        selected_doc_ids.add(doc_id)

# --- Step 3: Format and save the final selection ---
final_selection_df = df_hard.loc[list(selected_doc_ids)]

# Create the final list of dicts, including the rationale for classification
output_examples = []
for doc_id, row in final_selection_df.iterrows():
    # Generate a simple rationale for the Bedrock prompt
    rationale = ""
    if row['doc_label'] == 'conspiracy':
        rationale = "The text alleges a secret plot by a powerful group with a malicious goal."
    elif row['doc_label'] == 'non':
        rationale = "The text is critical but does not allege a secret, coordinated, malicious plot."
    else:
        rationale = "There is not enough information in the text to make a confident decision."

    output_examples.append({
        "doc_id": doc_id,
        "text": row['text'],
        "markers": row['markers'],
        "doc_label": row['doc_label'],
        "rationale": rationale
    })

```

```

# --- Save to a file ---
output_path = PIPELINE_OUTPUT_DIR / "best_fewshot_examples.json"
with open(output_path, 'w', encoding='utf-8') as f:
    json.dump(output_examples, f, indent=2, ensure_ascii=False)

print(f"\nSuccessfully selected {len(output_examples)} examples.")
print(f"Saved best examples to: {output_path}")

# Display the final selection for review
print("\nFinal Selected Examples:")
for ex in output_examples:
    print(json.dumps(ex, indent=2))

```

Successfully selected 3 examples.

Saved best examples to: C:\Users\panagiotis\Desktop\GitHub\PsychoMark_Semeval\data\derived\psycomark_official_split_20250928_232947\best_fewshot_examples.json

Final Selected Examples:

```

{
  "doc_id": "t1_f3otibj",
  "text": "I try not clog up this forum with stuff off r/gundeals anymore, but that's just a way low price. That's lower than a box of .22LR now. That's lower than the boxes of birdshot shotgun ammo Wal-Mart carries. I've never seen 9mm offered that low by a store chain before. \n Turner's doesn't have the greatest reputation, but just buy a couple of boxes if you can. A little is literally going a long way here.",
  "markers": [
    {
      "label": "Actor",
      "start": 188,
      "end": 196,
      "text": "Wal-Mart"
    },
    {
      "label": "Evidence",
      "start": 206,
      "end": 267,
      "text": "I've never seen 9mm offered that low by a store chain before."
    },
    {
      "label": "Action",
      "start": 222,
      "end": 242,
      "text": "9mm offered that low"
    }
  ]
}

```

```

        "label": "Effect",
        "start": 222,
        "end": 242,
        "text": "9mm offered that low"
    },
    {
        "label": "Actor",
        "start": 271,
        "end": 279,
        "text": "Turner's"
    }
],
"doc_label": "non",
"rationale": "The text is critical but does not allege a secret, coordinated,
malicious plot."
}
{
    "doc_id": "t1_f7bzvwr",
    "text": "World War One killed millions of people, destroyed all of the wealth
built up from the industrial revolution, Lead to the Balfour declaration, turned
Russia communist and endebted most nations in the world to the banks. \n And
Armistice day is literally written on the car in which Franzferdinand was
assassinated in 1914. The chances of that number plate combination being random
are over 1 in a billion.",
    "markers": [
        {
            "label": "Actor",
            "start": 0,
            "end": 5,
            "text": "World"
        },
        {
            "label": "Action",
            "start": 41,
            "end": 50,
            "text": "destroyed"
        },
        {
            "label": "Effect",
            "start": 41,
            "end": 50,
            "text": "destroyed"
        },
        {
            "label": "Victim",
            "start": 185,
            "end": 192,
            "text": "nations"
        }
    ]
}

```

```

    },
    {
      "label": "Evidence",
      "start": 253,
      "end": 260,
      "text": "written"
    }
  ],
  "doc_label": "conspiracy",
  "rationale": "The text alleges a secret plot by a powerful group with a
malicious goal."
}
{
  "doc_id": "t1_e1pv6xg",
  "text": "Amy Manson, actress in the upcoming Doom movie references the removal
of DoomGuy followed by a quick backtracking from Tony Gigilo attempting to
excuse it as a joke. Accidentally iterates that sexism and misandry is okay, and
a joke.",
  "markers": [
    {
      "label": "Actor",
      "start": 0,
      "end": 11,
      "text": "Amy Manson,"
    },
    {
      "label": "Evidence",
      "start": 47,
      "end": 80,
      "text": "references the removal of DoomGuy"
    },
    {
      "label": "Action",
      "start": 62,
      "end": 80,
      "text": "removal of DoomGuy"
    },
    {
      "label": "Effect",
      "start": 62,
      "end": 80,
      "text": "removal of DoomGuy"
    },
    {
      "label": "Victim",
      "start": 73,
      "end": 80,
      "text": "DoomGuy"
    }
  ]
}

```

```

    },
    {
      "label": "Action",
      "start": 93,
      "end": 113,
      "text": "a quick backtracking"
    },
    {
      "label": "Actor",
      "start": 120,
      "end": 131,
      "text": "Tony Gigilo"
    },
    {
      "label": "Effect",
      "start": 181,
      "end": 235,
      "text": "iterates that sexism and misandry is okay, and a joke."
    }
  ],
  "doc_label": "conspiracy",
  "rationale": "The text alleges a secret plot by a powerful group with a
malicious goal."
}

```

[]: